PYTHON

(PCEP STUDY NOTES)

COMPLETE NOTES

PCEP EXAM MAP

PCEP-30-02 is the active Python Institute entry-level exam: 30 questions, 40 exam minutes plus 5 minutes NDA/tutorial, 70% passing score, and four syllabus blocks.

TOPICS COVERED

* Computer programming fundamentals
* Python syntax and execution
* Literals, variables, and data types
* Operators and expressions
* Input, output, and type conversion
* if / elif / else control flow
* while and for loops
* Lists, tuples, dicts, strings
* Functions, arguments, return
* Exceptions and debugging

QUICK HIGHLIGHTS

* Code runs top to bottom
* Indentation defines blocks
* Everything has a type
* Lists are mutable
* Tuples are immutable
* Dicts map keys to values
* Functions package reusable logic
* Exceptions are handled with try/except
* Tracebacks are clues
* Practice beats memorizing

PCEP ROADMAP

Exam block	Topic	Weight
Block 1	Computer programming and Python fundamentals	18%
Block 2	Control flow: conditionals and loops	29%
Block 3	Data collections: lists, tuples, dictionaries, strings	25%
Block 4	Functions and exceptions	28%

HOW TO STUDY

- Read a concept.
- Type small examples.
- Predict output before running.
- Explain why it worked.

PRACTICE LOOP

- Write.
- Run.
- Read traceback.
- Fix one thing.
- Run again.

WATCH FOR

- Indentation.
- Off-by-one range errors.
- Mutable list changes.
- String/list index positions.

BASH ANALOGY

- Variables hold text or values.
- Loops repeat work.
- Functions are reusable command blocks.
- Exit errors map loosely to exceptions.

KEY POINT * PCEP is about reading and reasoning through small Python programs. Predict output, then run the code to test your mental model.

PYTHON MENTAL MODEL

INTERPRETED LANGUAGE

- Python reads and executes statements.
- Errors appear at runtime for many mistakes.
- The traceback points to where Python got stuck.

INDENTATION MATTERS

- Blocks are defined by leading spaces.
- Do not mix tabs and spaces.
- Indent after colon statements like if, for, while, def.

TINY PROGRAM

```
name = input('Name: ')
age = int(input('Age: '))

if age >= 18:
    print(name, 'can vote')
else:
    print(name, 'is under 18')
```

KEY POINT * Think like Bash plus stricter structure: Python variables name objects, indentation defines blocks, and exceptions replace many shell-style error checks.

LITERALS, VARIABLES, TYPES

Type	Example	Meaning
int	42	whole number
float	3.14	decimal number
str	'router'	text
bool	True / False	logic value
NoneType	None	no value

VARIABLES

```
hostname = 'r1'
metric = 10
enabled = True

print(type(hostname))
print(type(metric))
```

NAMING RULES

- Use letters, numbers, and underscore.
- Name cannot start with a number.
- Names are case-sensitive.
- Avoid Python keywords.
- Prefer clear snake_case names.

TYPE CONVERSION

- int('10') -> 10
- str(10) -> '10'
- float('3.5') -> 3.5
- bool('text') -> True

COMMON ERROR

- '10' + 5 fails.
- Convert first: int('10') + 5.
- input() always returns a string.

KEY POINT * A variable is a name pointing at a value. Always know the value's type before using an operator.

OPERATORS + EXPRESSIONS

Category	Operators	Meaning
Arithmetic	+ - * / // % **	math operations
Comparison	== != < <= > >=	returns bool
Logical	and or not	combine bool values
Assignment	= += -= *= /=	update a name
Membership	in / not in	collection contains value?
Identity	is / is not	same object?

PRECEDENCE EXAMPLE

```
result = 2 + 3 * 4
print(result)      # 14

result = (2 + 3) * 4
print(result)      # 20

print(10 // 3)     # 3
print(10 % 3)      # 1
```

WATCH OUT

- = assigns; == compares.
- / returns float division.
- // returns floor division.
- ** exponentiation binds tightly.
- and/or short-circuit left to right.

KEY POINT * When an expression looks tricky, add parentheses. The exam often checks whether you know precedence and integer vs float division.

INPUT, OUTPUT, STRINGS

INPUT / OUTPUT

```
name = input('Hostname: ')
count = int(input('Count: '))

print('Device:', name)
print(f'{name} has {count} links')
```

STRING BASICS

```
text = 'OSPF Area 0'
print(text[0])       # O
print(text[-1])      # 0
print(text[0:4])     # OSPF
print(text.lower())
print(text.split())
```

String tool	Use
len(s)	length
s.strip()	remove edge whitespace
s.lower()	lowercase copy
s.upper()	uppercase copy
s.find('x')	index or -1
s.replace(a,b)	new replaced string

STRING GOTCHA

- Strings are immutable.
- Methods return new strings.
- Indexes start at 0.
- Slices stop before the end index.

FORMAT GOTCHA

- f-strings are readable.
- Commas in print add spaces.
- + requires strings on both sides.

KEY POINT * input() returns text. Convert it when you need math, and leave it as text when you need string work.

CONDITIONAL LOGIC

IF / ELIF / ELSE

```
score = 82

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
else:
    grade = 'C or lower'

print(grade)
```

CONDITION RULES

- A condition must evaluate to True or False.
- Colon starts the block.
- Indent all statements inside the block.
- elif checks only if previous tests failed.
- else catches everything remaining.

Concept	Rule
False-like	False, None, 0, 0.0, '', [], (), {}
True-like	Most non-empty and non-zero values
and	True only if both sides are true
or	True if either side is true
not	Flips truth value

KEY POINT * Python decides blocks by indentation. If the indent is wrong, your logic is wrong even when the condition text looks right.

LOOPS

FOR LOOP

```
devices = ['r1', 'r2', 'r3']

for device in devices:
    print(device)

for i in range(3):
    print(i)        # 0, 1, 2
```

WHILE LOOP

```
retries = 3

while retries > 0:
    print('try')
    retries -= 1

print('done')
```

Tool	Meaning
break	Exit the loop immediately
continue	Skip to next iteration
else on loop	Runs only if loop did not break
range(stop)	0 through stop-1
range(start, stop, step)	custom start/stop/step

COMMON BUG

- Off-by-one with range.
- Infinite while loop.
- Changing a list while iterating.
- Indenting print outside the loop.

NETWORK ANALOGY

- Loop through device list.
- Retry command until success.
- Skip unreachable host with continue.
- Stop scan early with break.

KEY POINT * for loops are for known collections/ranges. while loops are for repeat-until-a-condition-changes.

LISTS

LIST BASICS

```
vlans = [10, 20, 30]
vlans.append(40)
vlans[1] = 200

print(vlans[0])     # 10
print(vlans[-1])    # 40
print(vlans[1:3])   # [200, 30]
```

LIST METHODS

- append(x): add to end.
- insert(i, x): add at index.
- pop(): remove and return.
- remove(x): remove first matching value.
- sort(): sort in place.
- reverse(): reverse in place.

List property	Answer
Mutable?	Yes
Ordered?	Yes
Duplicates?	Yes
Index starts	0
Common error	IndexError

SLICE RULE

- items[start:stop]
- start is included.
- stop is excluded.
- missing start/end means beginning/end.

COPY GOTCHA

- a = b points at same list.
- b = a[:] makes shallow copy.
- Mutating one shared list affects both names.

KEY POINT * Lists are mutable. That means methods can change the list in place, which is powerful and exam-friendly.

TUPLES, DICTS, STRINGS

Collection	Example	Trait
Tuple	('r1', '10.0.0.1')	Ordered, immutable
Dictionary	{'r1': '10.0.0.1'}	Key/value mapping
String	'router'	Ordered, immutable text
List	['r1', 'r2']	Ordered, mutable

DICTIONARY

```
device = {'name': 'r1', 'asn': 65000}
print(device['name'])
device['role'] = 'edge'

for key, value in device.items():
    print(key, value)
```

TUPLE UNPACKING

```
point = ('r1', '10.0.0.1')
name, ip = point

print(name)
print(ip)

letters = tuple('abc')
```

DICT GOTCHA

- Missing key raises KeyError.
- Use get() when absence is expected.
- Keys must be hashable.

TUPLE GOTCHA

- Tuple item cannot be reassigned.
- Single-item tuple needs comma: (1,).
- Tuples are often used for records.

KEY POINT * Choose the collection by behavior: list for changeable sequence, tuple for fixed record, dict for lookup by key, string for text.

FUNCTIONS

DEFINE + CALL

```
def greet(name):
    return f'Hello {name}'

message = greet('Varun')
print(message)

def log(msg='ok'):
    print(msg)

log()
```

FUNCTION VOCAB

- def creates a function.
- Parameters are names in the function definition.
- Arguments are values passed during the call.
- return sends a value back.
- No return means the function returns None.

Argument type	Example	Meaning
Positional	func(1, 2)	Matched by order
Keyword	func(port=22)	Matched by name
Default	def f(x=1)	Used if caller omits value
Return	return value	Exits function and sends value

GOOD HABIT

- One clear job per function.
- Return values instead of only printing.
- Use meaningful parameter names.

EXAM GOTCHA

- Default arguments are evaluated when function is defined.
- return stops the function immediately.
- Code after return does not run.

KEY POINT * A function is a reusable mini-script with inputs, local work, and often one output.

SCOPE

LOCAL VS GLOBAL

```
x = 10

def demo():
    x = 5
    print(x)   # 5

demo()
print(x)       # 10
```

GLOBAL KEYWORD

```
count = 0

def add_one():
    global count
    count += 1

add_one()
print(count)
```

LOCAL

- Created inside a function.
- Visible only inside that function.
- Destroyed after function returns.

GLOBAL

- Created outside functions.
- Readable inside functions.
- Avoid changing it unless necessary.

SHADOWING

- Local name can hide global name.
- Same name does not mean same variable.
- Use clear names to avoid confusion.

RETURN INSTEAD

- Prefer return values.
- Assign result outside the function.
- Keeps code easier to test.

KEY POINT * If a function assigns to a name, Python treats that name as local unless you explicitly declare otherwise.

EXCEPTIONS

TRY / EXCEPT

```
try:
    age = int(input('Age: '))
    print(100 / age)
except ValueError:
    print('Enter a number')
except ZeroDivisionError:
    print('Age cannot be zero')
else:
    print('No error')
finally:
    print('done')
```

COMMON EXCEPTIONS

- ValueError: valid type, bad value.
- TypeError: operation on wrong type.
- IndexError: list index missing.
- KeyError: dict key missing.
- ZeroDivisionError: division by zero.
- NameError: unknown variable name.

Keyword	Meaning
try	Code that might fail
except	Handle a specific error
else	Runs if no exception happened
finally	Runs no matter what
raise	Create/throw an exception

KEY POINT * Do not catch everything blindly. Catch the exception you understand, then fix or report it clearly.

BUILT-INS + MODULES

Built-in	Use
len()	length of sequence/collection
type()	object type
int()/float()/str()	conversion
range()	integer sequence for loops
sorted()	new sorted list
sum()/min()/max()	numeric/collection helpers
print()/input()	console I/O

IMPORT MODULE

```
import math

print(math.sqrt(16))

from random import randint
print(randint(1, 6))
```

MODULE IDEA

- A module is a Python file/library.
- import makes its names available.
- Use module.name after plain import.
- Standard library modules ship with Python.
- PCEP focuses on basics, not deep packages.

KEY POINT * Built-ins are always available. Modules must be imported before you use their functions.

DEBUGGING TRACEBACKS

READ FROM BOTTOM UP

```
Traceback (most recent call last):
  File "script.py", line 4, in <module>
    total = count + '5'
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

WHAT LINE?

- Find the file and line number.
- Read the exact failing statement.
- Do not guess before reading it.

WHAT ERROR?

- TypeError means incompatible operation.
- ValueError means conversion/value problem.
- NameError means undefined name.

WHAT VALUE?

- Print or inspect variable values.
- Check type() when confused.
- Trace backward to assignment.

FIX SMALL

- Change one thing.
- Run again.
- Repeat until clean.

BASH ANALOGY

- stderr tells you where command failed.
- Python traceback is stderr with a map.
- Exit/error is not shame; it is signal.

GOOD HABIT

- Keep examples tiny.
- Name variables clearly.
- Run code after each change.

KEY POINT * A traceback is a troubleshooting artifact. Treat it like show output: read the line, read the state, fix the cause.

PCEP TRICKY QUESTIONS

MUTABILITY

- list changes in place.
- string/tuple operations return new objects.
- a = b can share the same list.

SLICING

- stop index is excluded.
- negative indexes count from end.
- out-of-range slices do not fail like bad indexes.

BOOLEAN LOGIC

- and/or return one operand, not always True/False.
- not has high precedence.
- empty collections are false-like.

RANGE

- range(3) gives 0,1,2.
- range(1,5,2) gives 1,3.
- stop value is not included.

FUNCTIONS

- default values exist before the call.
- return exits immediately.
- None appears when no value is returned.

EXCEPTIONS

- Order specific except blocks before broad ones.
- else runs only with no exception.
- finally always runs.

OUTPUT QUESTIONS

- Trace values line by line.
- Write the value beside each variable.
- Track indentation and loop count.

DO NOT OVERTHINK

- PCEP code snippets are small.
- Use the simplest language rule.
- Eliminate impossible answers first.

KEY POINT * Most PCEP misses are not advanced Python. They are indexing, indentation, type conversion, scope, or loop counting mistakes.

PRACTICE EXERCISES

EXERCISE 1 - PARSE VLANS

```
raw = '10,20,30'
vlans = raw.split(',')
for vlan in vlans:
    print(int(vlan))
```

EXERCISE 2 - FILTER

```
nums = [1, 2, 3, 4, 5]
even = []
for n in nums:
    if n % 2 == 0:
        even.append(n)
print(even)
```

EXERCISE 3 - FUNCTION

```
def prefix_ok(prefix):
    return '/' in prefix

print(prefix_ok('10.0.0.0/24'))
print(prefix_ok('10.0.0.0'))
```

EXERCISE 4 - EXCEPTION

```
try:
    mtu = int('nine thousand')
except ValueError:
    mtu = 1500

print(mtu)
```

HOW TO USE THIS PAGE

- Predict output first.
- Run the code second.
- Change one value and predict again.
- Explain it out loud in plain English.

STRETCH

- Turn raw VLAN text into a list of ints.
- Count words in a sentence.
- Build a dict of hostnames to IPs.
- Handle bad user input.

KEY POINT * The fastest way to learn Python is tiny scripts that do one real thing and fail in understandable ways.

NETWORK AUTOMATION MINI-PROJECT

READ A DEVICE LIST

```
devices = ['r1', 'r2', 'sw1']

def make_command(device):
    return f'show ip interface brief  # {device}'

for device in devices:
    command = make_command(device)
    print(command)
```

PCEP CONCEPTS USED

- list
- for loop
- function
- return
- f-string
- variable assignment

NEXT VERSION

- Read from a text file.
- Skip blank lines.
- Validate hostnames.
- Write commands to CSV.

ERROR HANDLING

- Bad filename -> FileNotFoundError.
- Bad input -> ValueError.
- Empty list -> print useful message.

WHY IT MATTERS

- Same fundamentals scale into real automation.
- Small loops become inventory workflows.
- Functions keep repeated logic tidy.

KEY POINT * PCEP basics are the same building blocks used in network automation: lists of devices, loops, parsing, functions, and clear errors.

EXAM TACTICS + STUDY PLAN

Stage	Focus	Practice
Week 1	Syntax, types, operators	Write 10 tiny scripts
Week 2	if/loops/range	Predict loop output
Week 3	lists/tuples/dicts/strings	Practice indexing/slicing
Week 4	functions/exceptions	Refactor scripts into functions
Final	Mixed review	Timed question sets

DURING QUESTIONS

- Read every line.
- Track variable values.
- Count loop iterations.
- Watch types after input/conversion.
- Check indentation.

ELIMINATION

- Remove answers with wrong type.
- Remove off-by-one outputs.
- Remove answers that ignore mutation.
- Compare exception order.

TEST DAY

- Do not race.
- Mark unsure questions.
- Use scratch notes for loops.
- Trust simple rules over vibes.

AFTER PRACTICE

- Review every miss.
- Rewrite the code from memory.
- Make one similar example.
- Run it and compare.

KEY POINT * Study by predicting outputs, not just reading explanations. PCEP is a reasoning exam.

ONE-PAGE CHEAT SHEET

Area	Remember
Types	int, float, str, bool, None
Operators	+ - * / // % **, == != < <= > >=
Truth	False, None, 0, '', [], {}, () are false-like
Flow	if / elif / else, while, for, break, continue
List	mutable, indexed, append/pop/sort
Tuple	immutable, ordered
Dict	key/value, get(), keys(), values(), items()
String	immutable, slicing, split/strip/replace
Function	def, parameters, arguments, return, None
Exception	try, except, else, finally, raise

MEMORIZE

- Indexes start at 0.
- Slice stop is excluded.
- input() returns str.
- = assigns, == compares.
- Indentation defines blocks.

PRACTICE

- Predict output.
- Run code.
- Read traceback.
- Fix one thing.
- Explain why.

Before scheduling, re-check the current Python Institute PCEP exam page for version and syllabus.

KEY POINT * PCEP is the foundation. Get types, loops, collections, functions, and exceptions solid, and real automation gets much easier.